SIMATS ENGINEERING

CO3-ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks **COURSE CODE:CSA07**

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4 **Total Marks:40**

Code of Conduct:

I Anand Paul(192425207) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

CASE STUDY 1 — Video Conferencing Quality Issues

Given Conditions

- Packet loss = **8%**
- Jitter = **180 ms**
- Bandwidth utilization = **fluctuating**
- Symptoms = voice distortion, frozen video, delayed communication

1. Analyze whether the problem is primarily caused by TCP/UDP or the application layer.

The problem is primarily related to the **transport protocol and network conditions**, with the **application-layer buffering strategy** also contributing to the poor quality.For real-time video conferencing, **UDP** is generally preferred because it provides low-delay transmission without waiting for retransmission of lost packets. However, UDP itself does not guarantee delivery, ordering, or congestion control.The observed **8% packet loss and 180 ms jitter** are very high for real-time communication. Packet loss causes missing audio/video data, while high jitter causes packets to arrive at irregular intervals, resulting in frozen frames and distorted audio.If the application uses excessive buffering to compensate for jitter, it can further increase communication delay. Therefore, the issue is a combination of network/transport problems and application-level buffering.

2. Justify based on real-time multimedia requirements.

Real-time multimedia applications prioritize **low latency, low jitter, and continuous playback** over perfect packet delivery.

Requirement	**Effect of current condition**
Low packet loss	8% loss causes missing audio/video
Low jitter	180 ms jitter causes uneven playback
Low latency	Delayed packets affect conversations
Stable bandwidth	Fluctuation causes video quality changes
Fast recovery	Retransmitting old packets may increase delay

UDP is suitable because it avoids retransmission delays. In contrast, **TCP guarantees reliable and ordered delivery**, but retransmission and congestion control can introduce additional delay and head-of-line blocking.

For a live meeting, receiving a packet slightly late can be almost as bad as losing it. Therefore, **low latency is more important than perfect reliability**.

3. Recommended improvements

Transport/Network-level improvements

1. Prefer **UDP-based real-time transport** such as RTP/RTCP for media streams.
2. Apply **QoS prioritization** to voice and video traffic.
3. Use congestion-control mechanisms suitable for real-time media.
4. Improve bandwidth capacity during peak hours.
5. Monitor and reduce packet loss and jitter.
6. Use forward error correction where appropriate.

Application-level improvements

1. Use **adaptive bitrate (ABR)** to dynamically reduce video quality when bandwidth decreases.
2. Optimize the **jitter buffer** instead of using an excessively large buffer.
3. Use packet-loss concealment for missing audio/video packets.
4. Dynamically adjust resolution and frame rate.
5. Use silence suppression for audio where appropriate.

Conclusion

The major problem is the combination of **high packet loss, excessive jitter, unstable bandwidth, and buffering behavior**. A UDP-based real-time media approach with **QoS, adaptive bitrate, optimized jitter buffering, and congestion control** would significantly improve conferencing quality.

CASE STUDY 2 — Global Web Services and High DNS Lookup Time

Given Condition

- Average DNS lookup time = **800 ms**
- Target lookup time = **below 50 ms**
- Requirement = continuous availability during failures

1. Analyze the reasons for high DNS resolution time.

The high DNS resolution time may be caused by:

1. **Geographical distance** between users and DNS servers.
2. **Centralized DNS infrastructure**, causing excessive traffic at a few servers.
3. DNS queries requiring multiple recursive lookups.
4. Poor DNS caching configuration.
5. **Low or inappropriate TTL values**, causing frequent cache expiration.
6. Network congestion and packet loss.
7. Slow or overloaded authoritative DNS servers.
8. Lack of geographically distributed DNS infrastructure.

An average of **800 ms** indicates that DNS resolution has become a significant part of the application's page-load delay.

2. Proposed DNS architecture

A suitable architecture is a **globally distributed Anycast DNS system**.

Architecture

User → Nearest Anycast DNS Resolver → Cached DNS Response → Authoritative DNS Server

The company should deploy DNS servers at multiple geographical locations. All DNS servers can advertise the same **Anycast IP address** using BGP. Internet routing then directs users to a nearby/optimal DNS location.

DNS Pre-fetching

Frequently accessed domain records can be pre-fetched before their TTL expires. This reduces the probability of users experiencing cache misses.

TTL Optimization

Appropriate TTL values should be selected for different DNS records.

- Static records → longer TTL
- Frequently changing records → shorter TTL
- Critical records → balanced TTL for availability and freshness

Caching

Recursive DNS resolvers should cache frequently requested records. Cached responses can often be returned without contacting authoritative servers.

Target

The combination of **Anycast + caching + pre-fetching + optimized TTLs** can substantially reduce DNS lookup latency. The architecture should be monitored and tuned to achieve the organization's **<50 ms target**, although the exact value depends on user location, resolver behavior, and network conditions.

3. Advantages and limitations

Factor	**Advantages**	**Limitations**
Anycast DNS	Low latency, global distribution	More complex routing
DNS caching	Reduces repeated queries	Stale records possible
Pre-fetching	Reduces cache-miss delay	Additional DNS traffic
TTL optimization	Balances speed and freshness	Incorrect TTL can cause problems
Distributed servers	High availability	Higher infrastructure cost

Performance

Anycast directs users toward a nearby DNS service, while caching reduces repeated resolution. Therefore, DNS response time can be significantly reduced.

Scalability

Traffic is distributed across multiple DNS locations instead of one central server. This allows the infrastructure to handle millions of users.

Availability

If one DNS location fails, routing can direct queries toward another available location, reducing the possibility of a complete DNS outage.

Conclusion

A globally distributed Anycast DNS architecture combined with caching, DNS pre-fetching, and optimized TTL values is the most suitable solution. It improves performance, scalability, and fault tolerance while targeting DNS latency below 50 ms.

CASE STUDY 3 — Centralized DNS and Global Cloud Services

1. Examine the impact of centralized DNS architecture on application performance.

In a centralized DNS architecture, all DNS requests are directed to a **single primary DNS server**.

During promotional events, millions of users generate DNS queries simultaneously. This creates a bottleneck at the primary server.

The consequences include:

1. **Server overload**
2. Increased DNS response time
3. Longer application startup time
4. Increased network congestion
5. Single point of failure
6. Poor scalability during traffic spikes
7. Increased probability of DNS timeout

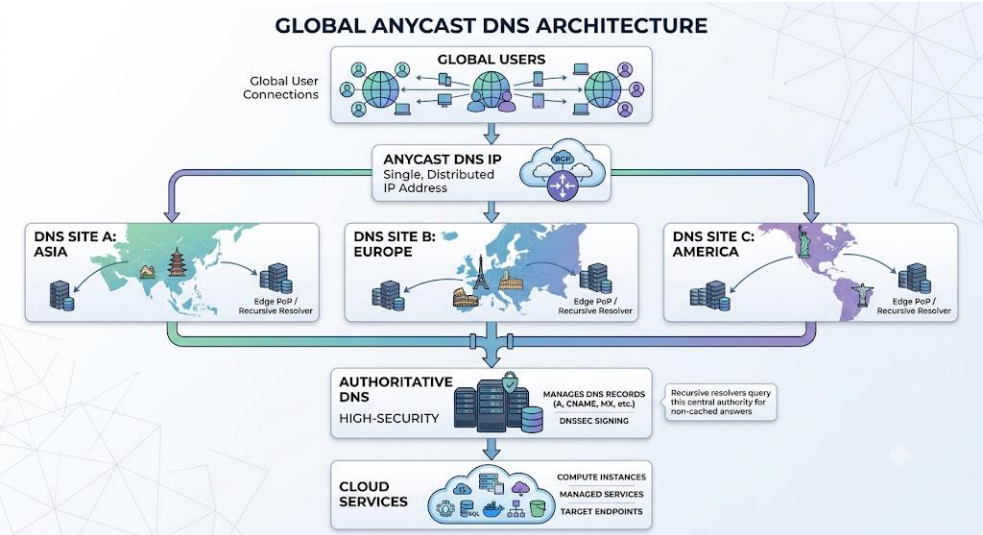
In this case, the **800 ms DNS lookup time** directly delays users before they can connect to the cloud application.

Therefore, centralized DNS negatively affects both **application performance and availability**.

2. Design a distributed DNS solution.

A suitable solution is a **globally distributed Anycast DNS architecture**.

Proposed design



Working

1. Users send DNS queries to the same Anycast IP address.
2. BGP routing directs each query toward a nearby/optimal DNS location.
3. Local DNS servers answer cached queries.
4. Cache misses are forwarded to authoritative DNS servers.
5. Multiple DNS locations provide redundancy.
6. If one location fails, routing directs traffic to another location.

Additional techniques

- **DNS caching** reduces repeated queries.
- **Appropriate TTL values** control cache lifetime.
- **DNS pre-fetching** can refresh popular records before expiration.
- Multiple authoritative servers eliminate a single point of failure.

Result

The architecture provides:

Lower latency + higher availability + better scalability + fault tolerance

3. Analyze TTL, Anycast routing, and DNS caching.

A. TTL

TTL (Time To Live) determines how long a DNS record can remain in a cache.

High TTL:

- Reduces DNS queries
- Improves performance
- Reduces server load
- But changes take longer to propagate

Low TTL:

- Faster propagation of changes
- More frequent DNS queries
- Higher DNS traffic and load

Therefore, an **optimized TTL** should be selected according to how frequently the DNS record changes.

B. Anycast Routing

Anycast allows multiple DNS servers in different locations to use the same IP address.

Advantages:

- Routes users to a nearby DNS server
- Reduces network latency
- Distributes traffic
- Provides redundancy
- Improves fault tolerance

If one DNS location becomes unavailable, routing can redirect users to another location.

C. DNS Caching

DNS caching stores previously resolved DNS records.

Benefits:

- Faster responses
- Reduced network traffic
- Lower authoritative-server load
- Better scalability

However, cached records may become **stale** until their TTL expires.

Conclusion

A combination of **Anycast routing, DNS caching, and optimized TTL values** removes the centralized bottleneck and provides a scalable and highly available DNS infrastructure.

CASE STUDY 4 — High-Frequency Stock Trading and TCP Latency

Given Conditions

- Normal acknowledgment latency = **1 ms**
- Peak latency = **40 ms**
- Increased TCP retransmissions
- Longer connection establishment
- Excessive buffering

1. Analyze three TCP-related factors

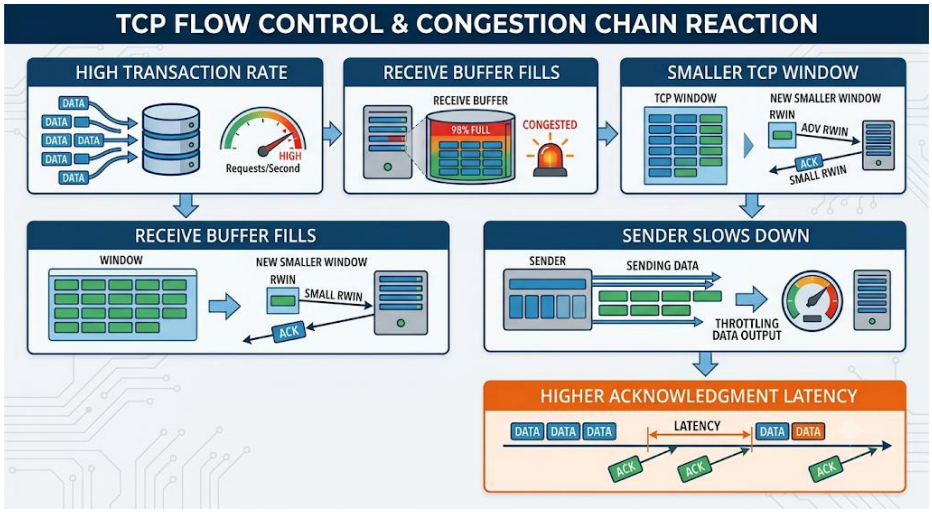
A. TCP Flow Control

TCP uses the **receive window** to prevent a sender from transmitting more data than the receiver can handle.

If the receiver's buffer becomes full, the advertised receive window decreases. The sender must slow down or temporarily stop transmission.

During high-frequency trading, thousands of small transactions can quickly fill buffers.

Effect:



Therefore, inefficient flow control or undersized buffers can contribute to increased latency.

B. Nagle's Algorithm

Nagle's algorithm combines small TCP packets into larger segments to improve network efficiency.

However, financial trading applications often send very small messages that require immediate transmission.

Nagle's algorithm may wait for an acknowledgment before sending additional small data.

This creates additional delay.

For latency-sensitive trading traffic, Nagle's algorithm can therefore increase transaction-processing latency.

Solution: Disable Nagle's algorithm using **TCP_NODELAY** where appropriate.

C. SYN Queue Limitations

TCP establishes connections using the **three-way handshake**:

Client → SYN → Server

Client ← SYN-ACK ← Server

Client → ACK → Server

During market opening, a very large number of clients may establish connections simultaneously.

If the server's **SYN queue/backlog** becomes full:

- New connection requests may be dropped.
- Retransmissions increase.
- Connection establishment takes longer.
- Application transactions are delayed.

Therefore, insufficient SYN backlog capacity can contribute to the observed latency increase.

2. Effect on transaction processing

TCP Factor	**Effect on trading system**
Flow control	Restricts sending rate when receiver buffers are full
Nagle's algorithm	Can delay small transaction messages
SYN queue limitations	Delays new TCP connections
Retransmissions	Increase latency and consume bandwidth
Excessive buffering	Causes queueing delay

In high-frequency trading, even a few milliseconds can be significant because transactions require **fast order submission and acknowledgment**.

3. Recommended TCP configuration changes

1. Disable Nagle's algorithm

Use:

TCP_NODELAY

This allows small trading messages to be transmitted immediately instead of waiting for aggregation.

2. Optimize TCP buffers

Configure suitable **send and receive buffer sizes** based on traffic characteristics.

Avoid both:

- undersized buffers → unnecessary throttling
- excessively large buffers → excessive queueing/bufferbloat

3. Increase SYN backlog capacity

Increase the server's TCP connection backlog/SYN queue capacity to handle large connection bursts during market opening.

4. Reduce unnecessary connection establishment

Use **persistent TCP connections or connection pooling** where the application architecture permits it.

This avoids repeatedly performing the TCP three-way handshake.

5. Investigate packet loss

Because retransmissions are already increasing, identify and eliminate:

- network congestion
- overloaded interfaces
- packet drops
- faulty links

6. Use appropriate congestion-control configuration

Select and tune a suitable TCP congestion-control algorithm for the environment while monitoring latency, retransmissions, and throughput.

7. Monitor continuously

Monitor:

- RTT

- retransmission rate
- TCP window size
- SYN backlog usage
- send/receive buffer utilization
- packet loss
- acknowledgment latency

Conclusion

The increase from 1 ms to 40 ms is likely caused by a combination of TCP flow-control limitations, Nagle-induced delays, SYN queue exhaustion, retransmissions, and excessive buffering. For a latency-sensitive trading platform, TCP_NODELAY, optimized buffers, sufficient SYN backlog, persistent connections, and reduction of packet loss can significantly improve transaction latency and reliability.